

Aula 4 — Entity Framework Core

Carga horária: 4 horas

Projeto base: Aula01.Web

Objetivo da aula:

Compreender o funcionamento de um **ORM moderno**, dominar o **Entity Framework Core**, configurar **DbContext**, executar **migrations**, entender **tracking vs no-tracking**, modelar **relacionamentos** e implementar um **CRUD persistente** com **SQL Server** ou **PostgreSQL**.

PARTE 1 — TEORIA (≈ 2 horas)

1 ORM vs SQL Puro

◆ SQL Puro

- ✓ Controle total
- ✓ Alta performance em casos específicos
- ✗ Mais código repetitivo
- ✗ Maior chance de erro

Exemplo:

```
SELECT * FROM Produtos WHERE Ativo = 1;
```

◆ ORM (Entity Framework Core)

- ✓ Abstração do banco
- ✓ Produtividade

- ✓ Integração com LINQ
- ✓ Segurança contra SQL Injection

Exemplo:

```
var produtos = context.Produtos.Where(p => p.Ativo).ToList();
```

👉 **Uso profissional:**

90% dos sistemas corporativos usam ORM + SQL pontual quando necessário.

2 DbContext

O DbContext :

- Representa a **sessão com o banco**
- Controla entidades
- Gerencia transações
- Executa consultas

```
public class AppDbContext : DbContext
```

3 Migrations

Migrations permitem:

- Versionar o banco
- Evoluir o schema
- Reproduzir ambientes

Comandos:

```
dotnet ef migrations add InitialCreate dotnet ef database update
```

4 Tracking vs NoTracking

◆ Tracking (padrão)

- EF monitora alterações
- Ideal para update

```
context.Produtos.First()
```

◆ NoTracking

- Mais performance
- Ideal para leitura

```
context.Produtos.AsNoTracking().ToList()
```

5 Relacionamentos

Tipos:

- 1:1
- 1:N
- N:N

Exemplo 1:N:

- Categoria → Produtos
-

PARTE 2 — PRÁTICA (≈ 2 horas)

1 Instalação dos Pacotes EF Core

SQL Server

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer dotnet add package Microsoft.EntityFrameworkCore.Tools
```

PostgreSQL

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
```

2 Configuração da Connection String

 appsettings.json

SQL Server

```
"ConnectionStrings": { "DefaultConnection":  
  "Server=localhost;Database=Aula01Db;Trusted_Connection=True;TrustServerCertificate=True" }
```

PostgreSQL

```
"ConnectionStrings": { "DefaultConnection": "Host=localhost;Port=5432;Database=aula01db;Username=postgres;Password=123456" }
```

3 Criando o DbContext

 Data/AppDbContext.cs

```
using Aula01.Web.Models; using Microsoft.EntityFrameworkCore; namespace Aula01.Web.Data { public class AppDbContext : DbContext {  
public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { } public DbSet<Produto> Produtos { get; set; }  
public DbSet<Categoria> Categorias { get; set; } protected override void OnModelCreating(ModelBuilder modelBuilder) { //  
Configuração de relacionamento 1:N modelBuilder.Entity<Categoria>().HasMany(c => c.Produtos).WithOne(p => p.Categoria)  
.HasForeignKey(p => p.CategoriaId); base.OnModelCreating(modelBuilder); } } }
```

4 Criando as Entidades com Relacionamento

 Models/Categoria.cs

```
namespace Aula01.Web.Models { public class Categoria { public int Id { get; set; } public string Nome { get; set; } // Navegação  
public List<Produto> Produtos { get; set; } } }
```

 Models/Produto.cs (atualizado)

```
using System.ComponentModel.DataAnnotations; namespace Aula01.Web.Models { public class Produto { public int Id { get; set; }  
[Required] public string Nome { get; set; } public decimal Preco { get; set; } public bool Ativo { get; set; } // FK public int  
CategoriaId { get; set; } public Categoria Categoria { get; set; } } }
```

5 Registro do DbContext no Program.cs

```
builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlServer(
builder.Configuration.GetConnectionString("DefaultConnection")) ); // Para PostgreSQL: // options.UseNpgsql(...)
```

6 Criando a Migration

```
dotnet ef migrations add InitialCreate dotnet ef database update
```

 Resultado:

```
Migrations/ └─ InitialCreate.cs └─ AppDbContextModelSnapshot.cs
```

7 Seed de Dados

 Data/DbInitializer.cs

```
using Aula01.Web.Models; namespace Aula01.Web.Data { public static class DbInitializer { public static void Seed(AppDbContext
context) { if (context.Categorias.Any()) return; var categorias = new List<Categoria> { new Categoria { Nome = "Eletrônicos" },
new Categoria { Nome = "Livros" } }; context.Categorias.AddRange(categorias); context.SaveChanges(); var produtos = new
List<Produto> { new Produto { Nome = "Notebook", Preco = 4500, Ativo = true, CategoriaId = categorias[0].Id }, new Produto { Nome
= "Livro C#", Preco = 120, Ativo = true, CategoriaId = categorias[1].Id } }; context.Produtos.AddRange(produtos);
context.SaveChanges(); } } }
```

 Chamar no Program.cs :

```
using (var scope = app.Services.CreateScope()) { var context = scope.ServiceProvider.GetRequiredService<AppDbContext>();
DbInitializer.Seed(context); }
```

8 Refatorando o CRUD para EF Core

Controllers/ProdutoController.cs

```
using Aula01.Web.Data; using Aula01.Web.Models; using Microsoft.AspNetCore.Mvc; using Microsoft.EntityFrameworkCore; namespace
Aula01.Web.Controllers { public class ProdutoController : Controller { private readonly ApplicationDbContext _context; public
ProdutoController(ApplicationDbContext context) { _context = context; } public async Task<IActionResult> Index() { var produtos = await
_context.Produtos .Include(p => p.Categoria) .AsNoTracking() .ToListAsync(); return View(produtos); } public IActionResult
Create() { ViewBag.Categorias = _context.Categorias.ToList(); return View(); } [HttpPost] public async Task<IActionResult>
Create(Produto produto) { if (!ModelState.IsValid) return View(produto); _context.Produtos.Add(produto); await
_context.SaveChangesAsync(); return RedirectToAction(nameof(Index)); } } }
```

9 View com Relacionamento

Views/Produto/Create.cshtml

```
@model Aula01.Web.Models.Produto <form asp-action="Create" method="post"> <input asp-for="Nome" /> <input asp-for="Preco" />
<select asp-for="CategoriaId" asp-items="@((new SelectList(ViewBag.Categorias, "Id", "Nome")))"> </select> <button
type="submit">Salvar</button> </form>
```

✓ Resultado Esperado

- ✓ Persistência real em banco
 - ✓ Migrations funcionando
 - ✓ Relacionamentos mapeados
 - ✓ Seed automático
 - ✓ CRUD profissional
-

Atividade Avaliativa

Desafio:

1. Criar entidade Pedido
2. Relacionar Pedido $\rightarrow$ Produto (N:N)
3. Criar migration
4. Exibir pedidos com produtos